Reviewer Pack — Minimal Order of Formal Language Automorphisms vs Monotone C…

Entry 0f8225fb42ac · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 15:31:56 UTC

Minimal Order of Formal Language Automorphisms vs Monotone Circuit Depth for k-CLIQUE

Entry ID: `0f8225fb42ac` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 15:31:56 UTC

1. Conjecture

Field A (mathematical branch): Formal Language Theory (Automata Theory) **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity

Statement:

> For all explicit functions in P, the minimal order of formal language automorphisms that preserve the k-CLIQUE problem has a monotone circuit depth of O(log n).

Rationale (proposer's reasoning):

> Formal language theory has historically provided insights into the structure of computation. If there is a connection between automorphism orders and circuit complexity for specific problems, it could lead to new algorithms or complexity lower bounds.

Taxonomy category: `FormalLanguageAutomorphismvsCircuitComplexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f2266858e4f7b152`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all explicit functions in P, the mean monotone circuit depth of formal language automorphisms is O(log n) with a standard deviation ≤ 0.5 log n across at least 10 seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `formal language theory AND automorphism AND k-CLIQUE AND monotone circuit complexity` - `automorphism order in P AND k-CLIQUE AND monotone circuit depth` - `k-CLIQUE problem AND formal language automorphisms AND monotone circuit depth O(log n)`

Top relevant hits considered: - [http://arxiv.org/abs/2406.09757v2] Evaluating LLM-driven User-Intent Formalization for Verification-Aware Languages - [http://arxiv.org/abs/1901.05180v1] Formal models of Structure Building in Music, Language and Animal Songs - [http://arxiv.org/abs/2503.21676v2] How do language models learn facts? Dynamics, curricula and hallucinations - [http://arxiv.org/abs/1511.09051v On automorphism groups of affine surfaces - [http://arxiv.org/abs/2501.09545v1] Hardness of clique approximation for monotone circuits - [http://arxiv.org/abs/2012.03883v2] Monotone Circuit Lower Bounds from Robust Sunflowers - [http://arxiv.org/abs/2310.14025v1] Large Language Models and Multimodal Retrieval for Visual Word Sense Disambiguation - [http://arxiv.org/abs/2105.04165v3] Inter-GPS: Interpretable Geometry Problem Solving with Formal Language and Symbolic Reasoning

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Define k-CLIQUE problem and related structures
    def generate_k_clique(n, k):
        if n < k:
            return None
        vertices = list(range(n))
        edges = []
        for i in range(k):
            for j in range(i + 1, k):
                edges.append((vertices[i], vertices[j]))
        return vertices, edges

    def is_clique(graph, clique):
        for u, v in itertools.combinations(clique, 2):
            if (u, v) not in graph or (v, u) not in graph:
                return False
        return True

    # Compute minimal order of formal language automorphisms
    def compute_automorphism_order(graph):
```

```
    n = len(graph)
    vertices = list(range(n))
    automorphisms = []

    def is_valid_permutation(perm):
        for u, v in graph:
            if perm[u] not in graph[perm[v]]:
                return False
        return True

    def generate_permutations():
        for p in itertools.permutations(vertices):
            if is_valid_permutation(p):
                automorphisms.append(p)

    generate_permutations()
    return len(automorphisms)

# Simulate on monotone circuit model and measure depth
def simulate_circuit_depth(graph, automorphism_order):
    n = len(graph)
    max_depth = 0

    def dfs(node, path, depth):
        nonlocal max_depth
        if depth > max_depth:
            max_depth = depth
        for neighbor in graph[node]:
            if neighbor not in path:
                dfs(neighbor, path + [neighbor], depth + 1)

    for start in range(n):
        dfs(start, [start], 0)

    return math.log2(max_depth) if max_depth > 0 else 0

# Main trial logic
n = random.randint(5, 40)
k = min(n, random.randint(3, 10))
graph = generate_k_clique(n, k)

if graph is None:
    return {
        "metric_name": "circuit_depth",
        "metric_value": 0,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "k_clique_not_possible"
    }

automorphism_order = compute_automorphism_order(graph)
circuit_depth = simulate_circuit_depth(graph, automorphism_order)

return {
    "metric_name": "circuit_depth",
    "metric_value": circuit_depth,
```

```
        "instances_tested": 1,
        "conjecture_holds": automorphism_order <= math.log2(n),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_depth = sum(r["metric_value"] for r in results) / len(results)
    std_depth = math.sqrt(sum((r["metric_value"] - mean_depth) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_depth} std={std_depth} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_depth} std={std_depth} support_fraction={support_frac
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_83ced768.py", line 107, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_83ced768.py", line 90, in run_trial
    automorphism_order = compute_automorphism_order(graph)
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_83ced768.py", line 55, in compute_autom
    generate_permutations()
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_83ced768.py", line 52, in generate_perm
    if is_valid_permutation(p):
       ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_83ced768.py", line 45, in is_valid_perm
    for u, v in graph:
        ^^^^
ValueError: too many values to unpack (expected 2)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means it did not complete its execution to provide the necessary results for evaluating the conjecture. | next: Investigate and fix the crash in the test code to ensure it can run to completion and produce the required data.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14913
2	propose	ollama_remote	glm4:latest	0	0	15380
3	propose	ollama_remote	glm4:latest	0	0	13351
4	preregistration	ollama_remote	glm4:latest	0	0	11036
5	novelty	ollama_remote	glm4:latest	0	0	8457
6	novelty	ollama_remote	glm4:latest	0	0	9927
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40319
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7214
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7583
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10652
11	judge	ollama_remote	glm4:latest	0	0	14048

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 152879 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/0f8225fb42ac.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0f8225fb42ac.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0f8225fb42ac.tar.gz` (if generated)